SMART PANTRY MANAGER

Mobile Application Development 700

Project Report

Student Name: Mcdonald Mudavanhu

Student Number: 402111443

Module: Mobile Application Development 700

Project: Smart Pantry Manager

Lecturer: Ben

Table of Content

Table of Content 1
1. Introduction 2
2. System Design 3
2.2 Screen Flow 4
2.3 Database Design and ERD 5
3. Application Implementation 6
4. Key Code Implementation 8
5. Challenges and Solutions 11
Conclusion and Reflection 12
7. References 13

1. Introduction

The Smart Pantry Manager is an Android application developed in Java to help users manage ingredients available in their pantry and identify recipes they can prepare using those ingredients. The application allows users to add, view, update, and delete pantry items while recording the quantity and unit of each ingredient.

A key feature of the application is its strict recipe-matching system. A recipe is suggested only when all its required ingredients are available in the pantry in sufficient quantities. The application also handles simple differences in ingredient names and measurement units to improve the accuracy of the matching process.

SQLite is used as the main database because it provides reliable local data storage without requiring an internet connection. The application also uses SharedPreferences to store user settings, such as the user's name and vegetarian preference. Other features include recipe details, preparation instructions, vegetarian filtering, and feedback when no recipes match the ingredients currently available.

The overall purpose of the project is to demonstrate practical Android development concepts, including Java programming, database management, CRUD operations, RecyclerView, multiple activities, user input validation, data persistence, and application navigation.

2. System Design

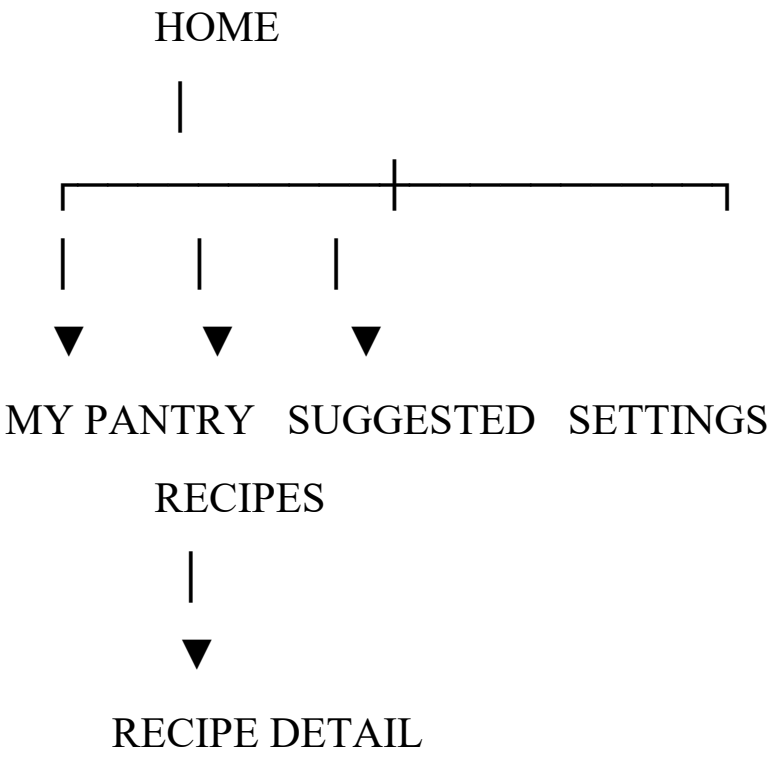
The Smart Pantry Manager was designed using multiple Android activities, with each activity responsible for a specific part of the application. The Home screen acts as the main entry point and allows the user to access the Pantry, Suggested Recipes, and Settings screens.

The Pantry screen manages the user's stored ingredients and provides CRUD functionality, allowing ingredients to be created, viewed, updated, and deleted. Pantry information is stored in an SQLite database so that the data remains available when the application is closed and reopened.

The Suggested Recipes screen retrieves the available pantry ingredients and compares them with the requirements of the recipes stored in the database. A recipe is displayed only when every required ingredient is available in a sufficient quantity. Users can then open the Recipe Detail screen to view the complete ingredient requirements and preparation instructions.

The Settings screen allows the user to save a name and vegetarian preference. These preferences are stored using SharedPreferences and are automatically loaded when the application is reopened.

2.2 Screen Flow



illustrates the main navigation flow of the Smart Pantry Manager application. The Home screen acts as the central starting point, allowing the user to access My Pantry, Suggested Recipes, or Settings. From the Suggested Recipes screen, the user can select an available recipe and open the Recipe Detail screen to view its required ingredients and preparation instructions. Back buttons are provided on the secondary screens to allow the user to return to the previous screen.

2.3 Database Design and ERD

Figure 2 presents the database structure used by the Smart Pantry Manager. The Pantry table stores the ingredients currently available to the user, together with their quantities and measurement units. The Recipe table stores the main information for each recipe, including its name, preparation instructions, and vegetarian classification.

The Recipe Ingredient table stores the ingredients required by each recipe. It has a many-to-one relationship with the Recipe table because one recipe can require several ingredients. Each required ingredient also stores its required quantity and unit.

During recipe matching, the application compares the records in the Pantry table with the required ingredients associated with each recipe. A recipe is suggested only when every required ingredient is available in a sufficient quantity.

3. Application Implementation

3.1 Home Screen

The Home screen is the main entry point of the Smart Pantry Manager application. It provides direct access to the three main areas of the system: My Pantry, Suggested Recipes, and Settings. The screen also displays a personalised welcome message using the name saved by the user in the Settings screen. Each button uses an Intent to open the corresponding Android activity.

3.2 Pantry Management

The My Pantry screen allows the user to manage the ingredients currently available in the pantry. The user enters an ingredient name, quantity and measurement unit before adding the item to the database. Existing ingredients are displayed using a RecyclerView connected to the SQLite database.

The screen supports full CRUD functionality. Users can create new ingredients, view stored ingredients, select and update existing records, and delete ingredients that are no longer available. Input validation is also used to prevent incomplete or invalid pantry records from being stored.

3.3 Suggested Recipes

The Suggested Recipes screen displays recipes that the user can prepare using the ingredients currently stored in the pantry. The application applies strict matching, meaning that every ingredient required by a

recipe must be available in a sufficient quantity before that recipe can be displayed.

The matching process compares ingredient names, quantities and measurement units. Simple singular and plural differences, such as “egg” and “eggs”, are normalised before comparison. The application also supports basic compatible unit conversions, such as kilograms to grams and litres to millilitres.

If one required ingredient is missing or its available quantity is too low, the recipe is excluded from the suggested results. This ensures that the application only recommends recipes that the user can actually prepare with the available pantry contents.

3.4 Recipe Detail

The Recipe Detail screen provides complete information about a recipe selected from the Suggested Recipes screen. It displays the recipe name, the required ingredients together with their quantities and units, and the preparation instructions.

The selected recipe information is passed from the Suggested Recipes screen to the Recipe Detail screen using an Android Intent. This allows the user to view the information needed to prepare the selected recipe on a separate, clear interface.

3.5 Settings and User Preferences

The Settings screen allows the user to personalise the application by entering a name and selecting whether only vegetarian recipes should be displayed. These preferences are stored using Android SharedPreferences.

SharedPreferences was suitable for this information because the settings consist of small key-value pairs and do not require relational database storage. Once saved, the preferences remain available after the application is closed and reopened. The saved name is also used to personalise the welcome message on the Home screen, while the vegetarian preference controls which recipes are displayed.

4. Key Code Implementation

4.1 SQLite Database

SQLite was used as the local database for the Smart Pantry Manager. The database helper class is responsible for creating and managing the tables required by the application. These tables store pantry ingredients, recipes, and the ingredients required by each recipe.

SQLite was selected because it is integrated with Android, supports structured relational data, works without an internet connection, and provides persistent storage. This means that pantry information remains available even after the application has been closed and reopened.

The database helper also supports the application's CRUD operations and provides the recipe data required by the strict matching process.

4.2 Pantry CRUD Operations

The Pantry screen implements CRUD operations to manage the user's ingredients. When an ingredient is added, the application first retrieves and validates the values entered by the user. Valid information is then passed to the database helper and stored in SQLite.

The application also allows existing pantry records to be retrieved, updated, and deleted. After a database operation is performed, the pantry list is refreshed so that the RecyclerView displays the latest information.

This provides immediate visual feedback to the user and keeps the interface consistent with the stored database records.

4.3 Strict Recipe Matching

The strict recipe-matching algorithm is responsible for deciding whether a recipe can be suggested. For each recipe, the application checks every required ingredient against the ingredients currently available in the pantry.

The algorithm does not consider a recipe suitable simply because some ingredients match. Every required ingredient must be found and the available quantity must be equal to or greater than the required quantity. If an ingredient is missing, has an incompatible unit, or has an insufficient quantity, the method returns false and the recipe is excluded. The method returns true only after all required ingredients successfully pass these checks.

This approach implements the main requirement of the Smart Pantry Manager by ensuring that suggested recipes can actually be prepared using the user's available ingredients.

4.4 Quantity and Unit Conversion

The application includes basic measurement conversion to improve the accuracy of recipe matching. Pantry ingredients and recipe requirements may use different but compatible units. For example, a user may store milk in litres while a recipe requires millilitres.

Before comparing quantities, the application normalises the measurement units and converts compatible values into the required unit. Supported conversions include kilograms and grams, litres and millilitres, and common cooking measurements such as cups,

tablespoons and teaspoons. If the units cannot be converted reliably, the ingredients are treated as incompatible rather than assuming that the quantities are equivalent.

This prevents incorrect recipe suggestions and allows the strict matching algorithm to compare ingredient quantities more accurately.

4.5 Data Persistence

The Smart Pantry Manager uses two forms of persistent storage. SQLite stores structured application data such as pantry ingredients and recipe information, while SharedPreferences stores small user settings such as the user's name and vegetarian preference.

SharedPreferences allows these settings to be stored as key-value pairs and retrieved when the application is opened again. Similarly, SQLite ensures that pantry records are not lost when the application is closed. Testing confirmed that the saved pantry ingredients and user preferences remained available after restarting the application.

Using SQLite and SharedPreferences for different types of information keeps the application's storage approach simple while ensuring that important user data persists between sessions.

5. Challenges and Solutions

Several challenges were encountered during the development of the Smart Pantry Manager. One of the main challenges was implementing strict recipe matching. Checking ingredient names alone was not sufficient because a recipe could appear even when the user did not have enough of a required ingredient. This was addressed by extending the matching process to compare both the presence and quantity of every required ingredient.

Another challenge involved differences in ingredient names and measurement units. For example, “egg” and “eggs” should represent the same ingredient, while 1 litre and 1000 millilitres should represent equivalent quantities. Simple name and unit normalisation methods were introduced together with basic measurement conversions to handle these cases more reliably.

Data persistence was also important because pantry information and settings should not disappear when the application is closed. SQLite was used to persist structured pantry and recipe information, while SharedPreferences was used for smaller user settings such as the user's name and vegetarian preference.

During development, changes to the user interface and project configuration occasionally caused Android Studio to display temporary code-analysis or indexing errors even though the Gradle build remained successful. The project was synchronised and Android Studio's caches were refreshed, after which the application was rebuilt and tested successfully.These challenges helped improve the reliability of the final application and provided practical experience with debugging, database management, Android persistence, and application logic.

Conclusion and Reflection

The Smart Pantry Manager successfully demonstrates the development of a database-driven Android application using Java. The final application allows users to manage pantry ingredients through CRUD operations, store data persistently using SQLite, and receive recipe suggestions based strictly on the ingredients and quantities currently available.

The project also demonstrates important Android development concepts, including multiple activities, Intents, RecyclerView, input validation, SharedPreferences, SQLite database management, and persistent data storage. The strict recipe-matching feature was particularly important because it required ingredient names, quantities, and measurement units to be considered before a recipe could be recommended.

Developing the application provided practical experience in connecting a mobile user interface to a database and implementing application logic based on stored data. Testing and debugging were also important parts of the development process, particularly when dealing with data persistence, measurement conversions, recipe filtering, and Android Studio configuration issues.

Overall, the project achieved its main objective of creating a functional Smart Pantry Manager that helps users keep track of available ingredients and identify recipes they can prepare. Future improvements could include expiry-date tracking, a larger recipe collection, more advanced measurement conversions, and an improved navigation interface.

7. References

Android Developers (2026) *Activities and the activity lifecycle*. Available at: https://developer.android.com/guide/components/activities/activity-lifecycle (Accessed: 28 October 2026).

Android Developers (2026) *Intents and intent filters*. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 28 October 2026).

Android Developers (2026) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 28 October 2026).

Android Developers (2026) *Save simple data with SharedPreferences*. Available at: https://developer.android.com/training/data-storage/shared-preferences (Accessed: 28 October 2026).

Android Developers (2026) *Create dynamic lists with RecyclerView*. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 28 October 2026).

SQLite (2026) *About SQLite*. Available at: https://www.sqlite.org/about.html (Accessed: 28 October 2026).